

Aggregation Pipeline Optimization

Aggregation pipeline operations have an optimization phase which attempts to reshape the pipeline for improved performance.

To see how the optimizer transforms a particular aggregation pipeline, include the `explain` option in the `db.collection.aggregate()` method.

Optimizations are subject to change between releases.

In addition to learning about the aggregation pipeline optimizations performed during the optimization phase, you will also see how to improve aggregation pipeline performance using indexes and document filters.



You can run aggregation pipelines in the UI for deployments hosted in MongoDB Atlas.

Projection Optimization

The aggregation pipeline can determine if it requires only a subset of the fields in the documents to obtain the results. If so, the pipeline only uses those fields, reducing the amount of data passing through the pipeline.

`$project` Stage Placement

When you use a `$project` stage it should typically be the last stage in your pipeline, used to specify which fields to return to the client.

Using a `$project` stage at the beginning or middle of a pipeline to reduce the number of fields passed to subsequent pipeline stages is unlikely to improve performance, as the database performs this optimization automatically.

Pipeline Sequence Optimization

`($project or $unset or $addFields or $set) + $match` Sequence Optimization

For an aggregation pipeline that contains a projection stage (`$addFields`, `$project`, `$set`, or `$unset`) followed by a `$match` stage, MongoDB moves any filters in the `$match` stage that do not require values computed in the projection stage to a new `$match` stage before the projection.

If an aggregation pipeline contains multiple projection or `$match` stages, MongoDB performs this optimization for each `$match` stage, moving each `$match` filter before all projection stages that the filter does not depend on.

Consider a pipeline with the following stages:

```
{
  $addFields: {
    maxTime: { $max: "$times" },
    minTime: { $min: "$times" }
  },
  {
    $project: {
      _id: 1,
      name: 1,
      times: 1,
      maxTime: 1,
      minTime: 1,
      avgTime: { $avg: ["$maxTime", "$minTime"] }
    }
  },
  {
    $match: {
      name: "Joe Schmo",
      maxTime: { $lt: 20 },
      minTime: { $gt: 5 },
      avgTime: { $gt: 7 }
    }
  }
}
```

The optimizer breaks up the `$match` stage into four individual filters, one for each key in the `$match` query document. The optimizer then moves each filter before as many projection stages as possible, creating new `$match` stages as needed.

Given this example, the optimizer automatically produces the following *optimized* pipeline:

```
{ $match: { name: "Joe Schmo" } },
{ $addFields: {
  maxTime: { $max: "$times" },
  minTime: { $min: "$times" }
} },
{ $match: { maxTime: { $lt: 20 }, minTime: { $gt: 5 } } },
{ $project: {
  _id: 1, name: 1, times: 1, maxTime: 1, minTime: 1,
  avgTime: { $avg: ["$maxTime", "$minTime"] }
} },
{ $match: { avgTime: { $gt: 7 } } }
```

NOTE

The optimized pipeline is not intended to be run manually. The original and optimized pipelines return the same results.

You can see the optimized pipeline in the explain plan.

The `$match` filter `{ avgTime: { $gt: 7 } }` depends on the `$project` stage to compute the `avgTime` field. The `$project` stage is the last projection stage in this pipeline, so the `$match` filter on `avgTime` could not be moved.

The `maxTime` and `minTime` fields are computed in the `$addField` stage but have no dependency on the `$project` stage. The optimizer created a new `$match` stage for the filters on these fields and placed it before the `$project` stage.

The `$match` filter `{ name: "Joe Schmoe" }` does not use any values computed in either the `$project` or `$addField` stages so it was moved to a new `$match` stage before both of the projection stages.

After optimization, the filter `{ name: "Joe Schmoe" }` is in a `$match` stage at the beginning of the pipeline. This has the added benefit of allowing the aggregation to use an index on the `name` field when initially querying the collection.

`$sort` + `$match` Sequence Optimization

When you have a sequence with `$sort` followed by a `$match`, the `$match` moves before the `$sort` to minimize the number of objects to sort. For example, if the pipeline consists of the following stages:

```
{ $sort: { age : -1 } },  
{ $match: { status: 'A' } }
```

During the optimization phase, the optimizer transforms the sequence to the following:

```
{ $match: { status: 'A' } },  
{ $sort: { age : -1 } }
```

`$redact` + `$match` Sequence Optimization

When possible, when the pipeline has the `$redact` stage immediately followed by the `$match` stage, the aggregation can sometimes add a portion of the `$match` stage before the `$redact` stage. If the added `$match` stage is at the start of a pipeline, the aggregation can use an index as well as query the collection to limit the number of documents that enter the pipeline. See [Improve Performance with Indexes](#) and [Document Filters](#) for more information.

For example, if the pipeline consists of the following stages:

```
{ $redact: { $cond: { if: { $eq: [ "$level", 5 ] }, then: "$$PRUNE",  
else: "$$DESCEND" } } },  
{ $match: { year: 2014, category: { $ne: "Z" } } }
```

The optimizer can add the same `$match` stage before the `$redact` stage:

```
{ $match: { year: 2014 } },  
{ $redact: { $cond: { if: { $eq: [ "$level", 5 ] }, then: "$$PRUNE",  
else: "$$DESCEND" } } },  
{ $match: { year: 2014, category: { $ne: "Z" } } }
```

`$project/$unset` + `$skip` Sequence Optimization

When you have a sequence with `$project` or `$unset` followed by `$skip`, the `$skip` moves before `$project`. For example, if the pipeline consists of the following stages:

```
{ $sort: { age : -1 } },  
{ $project: { status: 1, name: 1 } },  
{ $skip: 5 }
```

During the optimization phase, the optimizer transforms the sequence to the following:

```
{ $sort: { age : -1 } },  
{ $skip: 5 },  
{ $project: { status: 1, name: 1 } }
```

Pipeline Coalescence Optimization

When possible, the optimization phase coalesces a pipeline stage into its predecessor. Generally, coalescence occurs *after* any sequence reordering optimization.

`$sort` + `$limit` Coalescence

When a `$sort` precedes a `$limit`, the optimizer can coalesce the `$limit` into the `$sort` if no intervening stages modify the number of documents (e.g. `$unwind`, `$group`). MongoDB will not coalesce the `$limit` into the `$sort` if there are pipeline stages that change the number of documents between the `$sort` and `$limit` stages..

For example, if the pipeline consists of the following stages:

```
{ $sort : { age : -1 } },  
{ $project : { age : 1, status : 1, name : 1 } },  
{ $limit: 5 }
```

During the optimization phase, the optimizer coalesces the sequence to the following:

```
{  
  "$sort" : {  
    "sortKey" : {  
      "age" : -1  
    },  
    "limit" : NumberLong(5)  
  },  
  { "$project" : {  
    "age" : 1,  
    "status" : 1,  
    "name" : 1  
  }  
}
```

This allows the sort operation to only maintain the top `n` results as it progresses, where `n` is the specified limit, and MongoDB only needs to store `n` items in memory [1]. See `$sort` Operator and Memory for more information.

NOTE

Sequence Optimization with \$skip

If there is a `$skip` stage between the `$sort` and `$limit` stages, MongoDB will coalesce the `$limit` into the `$sort` stage and increase the `$limit` value by the `$skip` amount.

See `$sort + $skip + $limit` Sequence for an example.

[1] The optimization will still apply when `allowDiskUse` is `true` and the `n` items exceed the aggregation memory limit.

`$limit + $limit` Coalescence

When a `$limit` immediately follows another `$limit`, the two stages can coalesce into a single `$limit` where the limit amount is the *smaller* of the two initial limit amounts. For example, a pipeline contains the following sequence:

```
{ $limit: 100 },
{ $limit: 10 }
```



Then the second `$limit` stage can coalesce into the first `$limit` stage and result in a single `$limit` stage where the limit amount `10` is the minimum of the two initial limits `100` and `10`.

```
{ $limit: 10 }
```



`$skip + $skip` Coalescence

When a `$skip` immediately follows another `$skip`, the two stages can coalesce into a single `$skip` where the skip amount is the *sum* of the two initial skip amounts. For example, a pipeline contains the following sequence:

```
{ $skip: 5 },
{ $skip: 2 }
```



Then the second `$skip` stage can coalesce into the first `$skip` stage and result in a single `$skip` stage where the skip amount `7` is the sum of the two initial limits `5` and `2`.

```
{ $skip: 7 }
```



`$match + $match` Coalescence

When a `$match` immediately follows another `$match`, the two stages can coalesce into a single `$match` combining the conditions with an `$and`. For example, a pipeline contains the following sequence:

```
{ $match: { year: 2014 } },  
{ $match: { status: "A" } }
```

Then the second `$match` stage can coalesce into the first `$match` stage and result in a single `$match` stage

```
{ $match: { $and: [ { "year" : 2014 }, { "status" : "A" } ] } }
```

`$lookup`, `$unwind`, and `$match` Coalescence

When `$unwind` immediately follows `$lookup`, and the `$unwind` operates on the `as` field of the `$lookup`, the optimizer coalesces the `$unwind` into the `$lookup` stage. This avoids creating large intermediate documents. Furthermore, if `$unwind` is followed by a `$match` on any `as` subfield of the `$lookup`, the optimizer also coalesces the `$match`.

For example, a pipeline contains the following sequence:

```
{  
  $lookup: {  
    from: "otherCollection",  
    as: "resultingArray",  
    localField: "x",  
    foreignField: "y"  
  }  
},  
{ $unwind: "$resultingArray" },  
{ $match: {  
  "resultingArray.foo": "bar"  
}  
}
```

The optimizer coalesces the `$unwind` and `$match` stages into the `$lookup` stage. If you run the aggregation with `explain` option, the `explain` output shows the coalesced stages:

```
{  
  $lookup: {  
    from: "otherCollection",  
    as: "resultingArray",  
    localField: "x",  
    foreignField: "y",  
    let: {},  
    pipeline: [  
      {  
        $match: {  
          "foo": {
```

```

"$eq": "bar"
}
}
}
],
unwinding: {
  "preserveNullAndEmptyArrays": false
}
}
}

```

You can see this optimized pipeline in the explain plan.

Slot-Based Query Execution Engine Pipeline Optimizations

MongoDB can use the slot-based query execution engine to execute certain pipeline stages when specific conditions are met. In most cases, the slot-based execution engine provides improved performance and lower CPU and memory costs compared to the classic query engine.

To verify that the slot-based execution engine is used, run the aggregation with the `explain` option. This option outputs information on the aggregation's query plan. For more information on using `explain` with aggregations, see [Return Information on Aggregation Pipeline Operation](#).

The following sections describe:

- The conditions when the slot-based execution engine is used for aggregation.
- How to verify if the slot-based execution engine was used.

`$group` Optimization

New in version 5.2.

Starting in version 5.2, MongoDB uses the slot-based execution query engine to execute `$group` stages if either:

- `$group` is the first stage in the pipeline.
- All preceding stages in the pipeline can also be executed by the slot-based execution engine.

When the slot-based query execution engine is used for `$group`, the `explain` results include `queryPlanner.winningPlan.queryPlan.stage: "GROUP"`.

The location of the `queryPlanner` object depends on whether the pipeline contains stages after the `$group` stage that cannot be executed using the slot-based execution engine.

- If `$group` is the last stage or all stages after `$group` can be executed using the slot-based execution engine, the `queryPlanner` object is in the top-level `explain` output object (`explain.queryPlanner`).

- If the pipeline contains stages after `$group` that cannot be executed using the slot-based execution engine, the `queryPlanner` object is in `explain.stages[0].$cursor.queryPlanner`.

`$lookup` Optimization

New in version 6.0.

Starting in version 6.0, MongoDB can use the slot-based execution query engine to execute `$lookup` stages if *all* preceding stages in the pipeline can also be executed by the slot-based execution engine and none of the following conditions are true:

- The `$lookup` operation executes a pipeline on a joined collection. To see an example of this kind of operation, see [Join Conditions and Subqueries on a Joined Collection](#).
- The `$lookup`'s `localField` or `foreignField` specify numeric components. For example: `{ localField: "restaurant.0.review" }`.
- The `from` field of any `$lookup` in the pipeline specifies a view or sharded collection.

When the slot-based query execution engine is used for `$lookup`, the explain results include `queryPlanner.winningPlan.queryPlan.stage: "EQ_LOOKUP"`. `EQ_LOOKUP` means "equality lookup".

The location of the `queryPlanner` object depends on whether the pipeline contains stages after the `$lookup` stage that cannot be executed using the slot-based execution engine.

- If `$lookup` is the last stage or all stages after `$lookup` can be executed using the slot-based execution engine, the `queryPlanner` object is in the top-level `explain` output object (`explain.queryPlanner`).
- If the pipeline contains stages after `$lookup` that cannot be executed using the slot-based execution engine, the `queryPlanner` object is in `explain.stages[0].$cursor.queryPlanner`.

Improve Performance with Indexes and Document Filters

The following sections show how you can improve aggregation performance using indexes and document filters.

Indexes

An aggregation pipeline can use indexes from the input collection to improve performance. Using an index limits the amount of documents a stage processes. Ideally, an index can cover the stage query. A covered query has especially high performance, since the index returns all matching documents.

For example, a pipeline that consists of `$match`, `$sort`, `$group` can benefit from indexes at every stage:

- An index on the `$match` query field efficiently identifies the relevant data

- An index on the sorting field returns data in sorted order for the `$sort` stage
- An index on the grouping field that matches the `$sort` order returns all of the field values needed for the `$group` stage, making it a covered query.

To determine whether a pipeline uses indexes, review the query plan and look for `IXSCAN` or `DISTINCT_SCAN` plans.

NOTE

In some cases, the query planner uses a `DISTINCT_SCAN` index plan that returns one document per index key value. `DISTINCT_SCAN` executes faster than `IXSCAN` if there are multiple documents per key value. However, index scan parameters might affect the time comparison of `DISTINCT_SCAN` and `IXSCAN`.

For early stages in your aggregation pipeline, consider indexing the query fields. Stages that can benefit from indexes are:

`$match` stage

During the `$match` stage, the server can use an index if `$match` is the first stage in the pipeline, after any optimizations from the query planner.

`$sort` stage

During the `$sort` stage, the server can use an index if the stage is not preceded by a `$project`, `$unwind`, or `$group` stage.

`$group` stage

During the `$group` stage, the server can use an index to quickly find the `$first` or `$last` document in each group if the stage meets both of these conditions:

- The pipeline `sorts` and `groups` by the same field.
- The `$group` stage only uses the `$first` or `$last` accumulator operator.

See `$group` Performance Optimizations for an example.

`$geoNear` stage

The server always uses an index for the `$geoNear` stage, since it requires a geospatial index.

Additionally, stages later in the pipeline that retrieve data from other, unmodified collections can use indexes on those collections for optimization. These stages include:

- `$lookup`
- `$graphLookup`
- `$unionWith`

Document Filters

If your aggregation operation requires only a subset of the documents in a collection, filter the documents first:

- Use the `$match`, `$limit`, and `$skip` stages to restrict the documents that enter the pipeline.
- When possible, put `$match` at the beginning of the pipeline to use indexes that scan the matching documents in a collection.
- `$match` followed by `$sort` at the start of the pipeline is equivalent to a single query with a sort, and can use an index.

Example

`$sort` + `$skip` + `$limit` Sequence

A pipeline contains a sequence of `$sort` followed by a `$skip` followed by a `$limit`:

```
{ $sort: { age : -1 } },  
{ $skip: 10 },  
{ $limit: 5 }
```

The optimizer performs `$sort` + `$limit` Coalescence to transform the sequence to the following:

```
{  
  "$sort" : {  
    "sortKey" : {  
      "age" : -1  
    },  
    "limit" : NumberLong(15)  
  },  
  {  
    "$skip" : NumberLong(10)  
  }  
}
```

MongoDB increases the `$limit` amount with the reordering.

TIP

See also:

`explain` option in the `db.collection.aggregate()`